ERHVERVSAKADEMI KØBENHAVN

SQL Queries

Calculated Fields and Grouped Results

Today starts with examples from last time, then moves into one shared SQL dataset for calculated columns, aggregates, GROUP BY, and HAVING.

Show work | Shape output | Summarize rows

TODAY

What we will do

First review examples from last time. Then use one clean dataset to learn how SQL can shape rows and summarize groups.

Review | Calculated fields | Aggregation | Grouping

1. Review examples

A few scripts and ER diagrams are presented and discussed together.

2. Row-level output

Create display columns with aliases, CONCAT, TRIM, and arithmetic.

3. Summary values

Use COUNT, SUM, AVG, MIN, and MAX to summarize many rows.

4. Grouped results

Use GROUP BY, WHERE, HAVING, and ORDER BY to control grouped output.

REVIEW

Showcase and discussion

Each example gives us a way to compare design choices and script decisions.

- A few ER diagrams and database creation scripts are shown.
- We look for what is clear, what is sensible, and what could be improved.
- The goal is not to find every error. The goal is to learn from different solutions.
- At the end, we collect the main lessons and move into today's SQL topic.

Good discussion questions

What is the main idea of the model? Does the script match the design? Are the keys, names, and datatypes easy to understand?

Why this matters

Design choices affect how easy a database is to build, read, change, and query later.

REVIEW

What to look for in each example

ER logic — Are entities, attributes, and relationships understandable?

DDL match — Does the script reflect the same structure as the diagram?

Keys and types — Are primary keys clear and are datatypes reasonable?

Execution order — Could the script create the database and tables cleanly?

Naming — Do table and column names help a future reader?

Maintainability — Would another developer find it easy to extend later?

Keep returning to this question: Would another developer understand and trust this script?

From structure to output

Last time you designed and built a database. Today you will use SQL to make query results more readable and more meaningful.

Last time

ER ideas, schemas, DDL, and a script that can create a database.



This time

Calculated columns, aggregate functions, GROUP BY, HAVING, and sorted grouped results.

DATASET

Shared teaching table: orders

One table is enough for today because it supports row-level output, arithmetic, aggregates, and grouped results.

Column	Type	Purpose
order_id	INT PK	Unique row identifier
customer_first_name	VARCHAR(30)	First name
customer_last_name	VARCHAR(30)	Last name
city	VARCHAR(30)	Location for grouping
product_category	VARCHAR(30)	Category for grouping
product_name	VARCHAR(40)	Readable product
quantity	INT	Units in the order
unit_price	DECIMAL(8,2)	Price per unit
order_date	DATE	When the order was placed
status	VARCHAR(20)	Example: Completed or Pending

Why one table?

Today's challenge is not joins. Today's challenge is understanding the difference between one result per row and one result per group.

Useful patterns in the data

Repeated cities, repeated categories, different statuses, and numeric columns make grouping and aggregation easy to see.

CALCULATED FIELDS

Row-level output can be made more useful

A query does not have to return only stored columns. It can also create cleaner display columns without changing the stored data.

- Aliases rename output columns so results are easier to read.
- CONCAT combines values such as first name and last name.
- TRIM removes extra spaces from text values.
- Arithmetic creates new values from numeric columns.

Mental model

This is still row-by-row logic. Each input row produces one output row.

Important

Calculated fields change the result of the query, not the data stored in the table.

Stored data stays the same -> query output becomes easier to read

EXAMPLE

Aliases, CONCAT, and TRIM

SQL

```
SELECT
  CONCAT(TRIM(customer_first_name), ' ',
         TRIM(customer_last_name)) AS customer_name,
  city,
  product_name
FROM orders;
```

What this returns

A readable customer_name column, plus city and product_name for each row.

Why it matters

The table may store names in separate columns or with unwanted spaces. The query can still present clean output.

Question: does this change the table itself? No - it only changes the result set.

EXAMPLE

Arithmetic in the SELECT list

SQL

```
SELECT
  product_name,
  quantity,
  unit_price,
  quantity * unit_price AS line_total
FROM orders;
```

Result idea

Each row keeps its own product_name, quantity, and unit_price, then adds one new calculated column called line_total.

This is still not grouping. The query still returns one result per row.

AGGREGATION

Summary values change the level of the result

Aggregate functions do not keep one result per row. They summarize many rows into fewer values.

COUNT(*) How many rows are included?

SUM(...) What is the total?

AVG(...) What is the average?

MIN(...) What is the lowest value?

MAX(...) What is the highest value?

SQL

```
SELECT
  COUNT(*)                    AS completed_orders,
  SUM(quantity * unit_price) AS total_sales,
  AVG(quantity * unit_price) AS avg_order_value,
  MIN(quantity * unit_price) AS lowest_order,
  MAX(quantity * unit_price) AS highest_order
FROM orders
WHERE status = 'Completed';
```

Result shape

This query returns one summary row because it aggregates across all matching rows.

GROUP BY

From many rows to grouped summaries

Mental model

- Start with many rows.
- Put rows into groups that share the same value.
- Apply an aggregate inside each group.
- Return one row per group.

SQL

```
SELECT city,
       COUNT(*) AS
orders_per_city
FROM orders
GROUP BY city;
```

What changes?

Instead of one output row per order, the query now returns one output row per city.

Typical grouped columns

city, product_category, or status are good grouping columns because many rows can share the same value.

RULES

GROUP BY rules to remember

- Every selected column must either be aggregated or listed in GROUP BY.
- GROUP BY does not sort the result by itself. Use ORDER BY if you want sorting.
- Row-level thinking and group-level thinking are not the same thing.

Valid query

```
SELECT city, COUNT(*) AS orders_per_city
FROM orders
GROUP BY city;
```

Problem query

```
SELECT city, product_name, COUNT(*)
FROM orders
GROUP BY city;
```

Why the second query is a problem:

product_name is neither aggregated nor grouped, so SQL does not know which single product_name should represent each city.

FILTERING

WHERE vs HAVING

WHERE filters rows before grouping

```
SELECT city, COUNT(*) AS completed_orders
FROM orders
WHERE status = 'Completed'
GROUP BY city;
```

HAVING filters groups after grouping

```
SELECT city, COUNT(*) AS completed_orders
FROM orders
WHERE status = 'Completed'
GROUP BY city
HAVING COUNT(*) >= 2;
```

Use WHERE when...

the condition applies to original rows, such as status = 'Completed'.

Use HAVING when...

the condition depends on a whole group or an aggregate value.

SORTING GROUPED RESULTS

GROUP BY decides the groups. ORDER BY decides how the groups are shown.

SQL

```
SELECT city,
       COUNT(*) AS completed_orders,
       SUM(quantity * unit_price) AS total_sales
FROM orders
WHERE status = 'Completed'
GROUP BY city
ORDER BY total_sales DESC;
```

Reading the result

First build the groups, then sort the grouped output so the highest total_sales appears first.

Common mistake

Do not assume GROUP BY automatically sorts by the grouped column or by the aggregate.

COMBINED QUERY

WHERE, GROUP BY, HAVING, and ORDER BY together

SQL

```
SELECT city,
       COUNT(*) AS completed_orders,
       SUM(quantity * unit_price) AS total_sales
FROM orders
WHERE status = 'Completed'
GROUP BY city
HAVING SUM(quantity * unit_price) > 500
ORDER BY total_sales DESC;
```

- WHERE keeps only completed rows.
- GROUP BY creates one group per city.
- HAVING removes groups below the chosen threshold.
- ORDER BY shows the strongest group first.

PRACTICE

Work with the shared orders table

- Return customer_name, city, and product_name using CONCAT, TRIM, and aliases.
- Return product_name, quantity, unit_price, and a calculated line_total column.
- Show the number of completed orders per city.
- Show total completed sales per city and sort from highest to lowest.
- Show only cities whose completed sales are above a threshold using HAVING.

Goal of the practice

Move from row-level output to grouped summaries without changing the dataset or the business scenario.

Think before typing

Ask first: Do I want one result per row, or one result per group?

ERHVERVSAKADEMI KØBENHAVN

What to remember from today

- A query can create display columns without changing stored data.
- Aggregate functions summarize many rows into fewer values.
- GROUP BY changes the level of the result.
- WHERE filters rows before grouping.
- HAVING filters groups after grouping.
- ORDER BY still matters after grouping.

Simple mental check

Ask whether the query should return one row per original row or one row per group.

Next step later

The same ideas will matter again when you eventually group data across joined tables.